Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Docker

Build, Ship and Run Applications Anywhere

Containerise your applications for faster development and deployment.

Build Once | Run Anywhere | Deploy Faster | More Secure

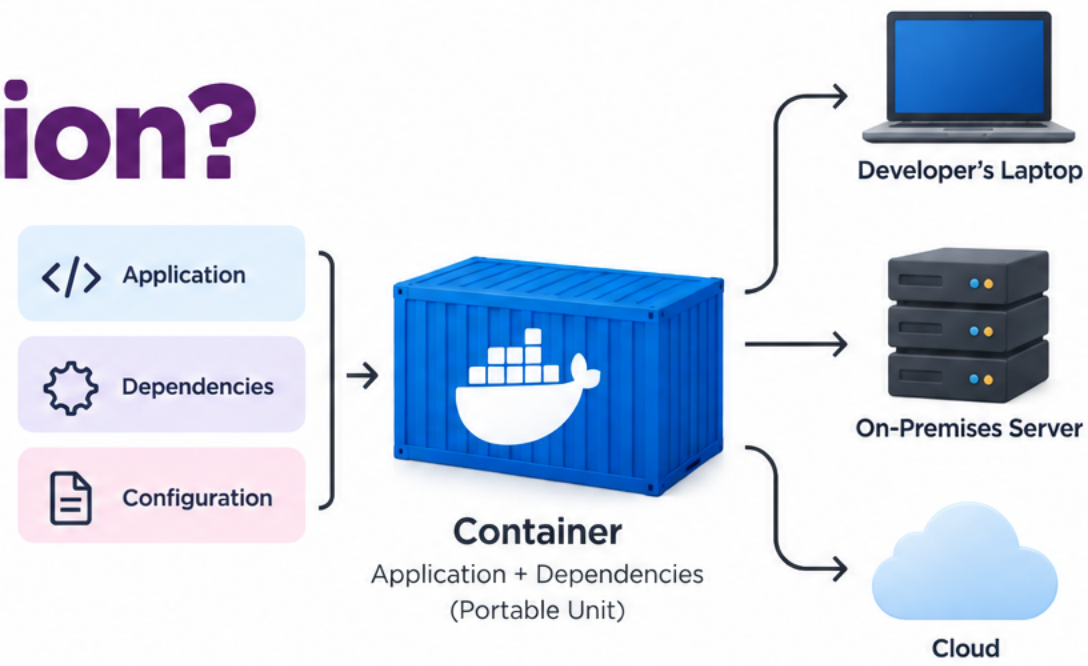
Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

What is Containerization?

Containerization is a method of packaging an application and all its dependencies (libraries, runtime, configuration, etc.) into a single, lightweight and portable unit called a **container**.

A container runs the same way on any environment, whether it is a developer's laptop, a test server or the cloud.



Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Problems Containers Solve

"It works on my machine"

Eliminates environment inconsistencies between development, testing and production.

Dependency conflicts

Packages applications with all required libraries and dependencies.

Complex deployments

Simplifies deployment and makes it faster and more reliable.

Poor scalability

Makes it easier to scale applications up or down based on demand.

Resource inefficiency

Containers are lightweight and use system resources more efficiently than virtual machines.

Slow and risky updates

Enables faster, safer and more consistent application updates and rollbacks.

Build Ship Run Anywhere

CONTAINERS FOR A BETTER TOMORROW

www.tech365.ng | +234 802 970 4730 | info@tech365.ng | @tech365ng | 4, Soji Adepegba Close, off Keystone Bank, Allen Avenue, Ikeja, Lagos.

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Traditional Deployment vs Virtual Machines vs Containers

Three different ways to run applications

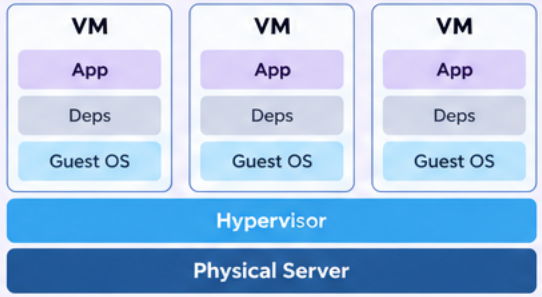
Traditional Deployment

One application per server

- Application is installed directly on a physical server.
- Each server usually runs one application.
- Managing multiple applications requires multiple servers.
- Lower resource efficiency.
- Difficult and slow to scale.

Virtual Machines (VMs)

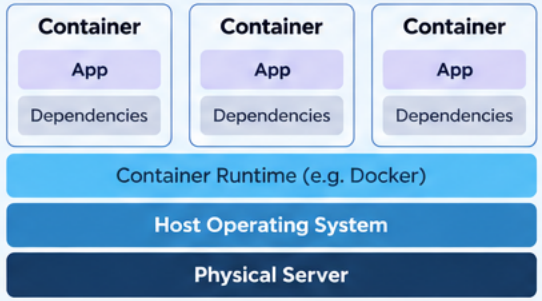
Multiple isolated environments on one server



- Each application runs in its own virtual machine.
- Each VM has its own operating system.
- Provides strong isolation.
- Uses more resources because each VM includes a full OS.
- Slower to start and scale compared to containers.
- Useful for running different operating systems on the same hardware.

Containers

Multiple isolated applications sharing the same OS



- Each application is packaged with its dependencies.
- Containers share the host operating system.
- Lightweight and fast to start.
- Uses fewer resources than VMs.
- Easier to deploy, scale and manage.
- Provides process-level isolation.
- Ideal for modern, cloud-native applications.

Key Takeaways

Traditional deployment is simple but not scalable.

Virtual machines provide strong isolation but are heavier and slower.

Containers are lightweight, faster and more efficient, making them ideal for modern application deployment.

www.tech365.ng | +234 802 970 4730 | info@tech365.ng | @tech365ng | 4, Soji Adepegba Close, off Keystone Bank, Allen Avenue, Ikeja, Lagos.

Tech365
Leveraging technology for growth

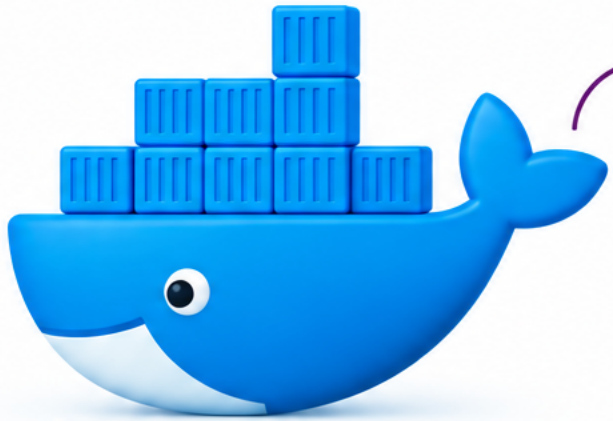
LEARN | PRACTICE | APPLY

What is Docker?

Docker is a platform that helps you build, ship and run applications inside **containers**.

A container packages your application and everything it needs (code, dependencies, runtime, configuration, etc.) into a single, lightweight and portable unit.

With Docker, you can run the same application consistently on your laptop, a test server or in the cloud.



- Package Your Application
- Run Consistently
- Deploy Anywhere
- Save Time and Resources

www.tech365.ng | +234 802 970 4730 | info@tech365.ng | @tech365ng

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Docker Use Cases

Docker is used in many real-world scenarios across development, testing and production.

Development Environments

Create consistent and isolated development environments for your team.

Application Deployment

Package and deploy applications easily to any environment.

Testing and CI/CD

Run automated tests in clean and consistent environments.

Microservices Architecture

Run and manage multiple services independently using containers.

Running Databases and Services

Quickly run databases such as PostgreSQL, MySQL, Redis and more in containers.

Cloud Deployment

Build once and deploy on any cloud platform (AWS, Azure, Google Cloud) with the same container.

Scalable Applications

Easily scale applications horizontally using container orchestration tools such as Kubernetes.

Consistent Environments

Ensure your application runs the same way on your laptop, a test server and in production.

docker®

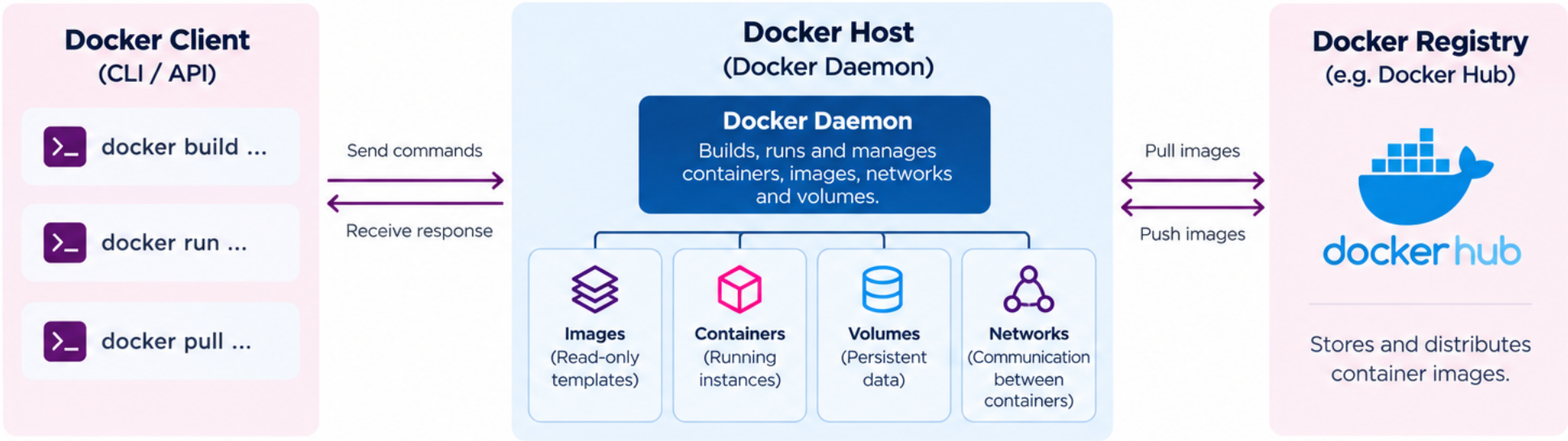
Build • Ship • Run Anywhere

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Docker Architecture

The main components that work together to build, ship and run containers.



1 Client

You interact with Docker using the command line (CLI) or an API.

2 Docker Daemon

A background service that handles all Docker operations on the host.

3 Docker Objects

Images, containers, volumes and networks are managed by the daemon.

4 Registry

A remote repository (such as Docker Hub) that stores container images which you can pull or push.

Docker Engine

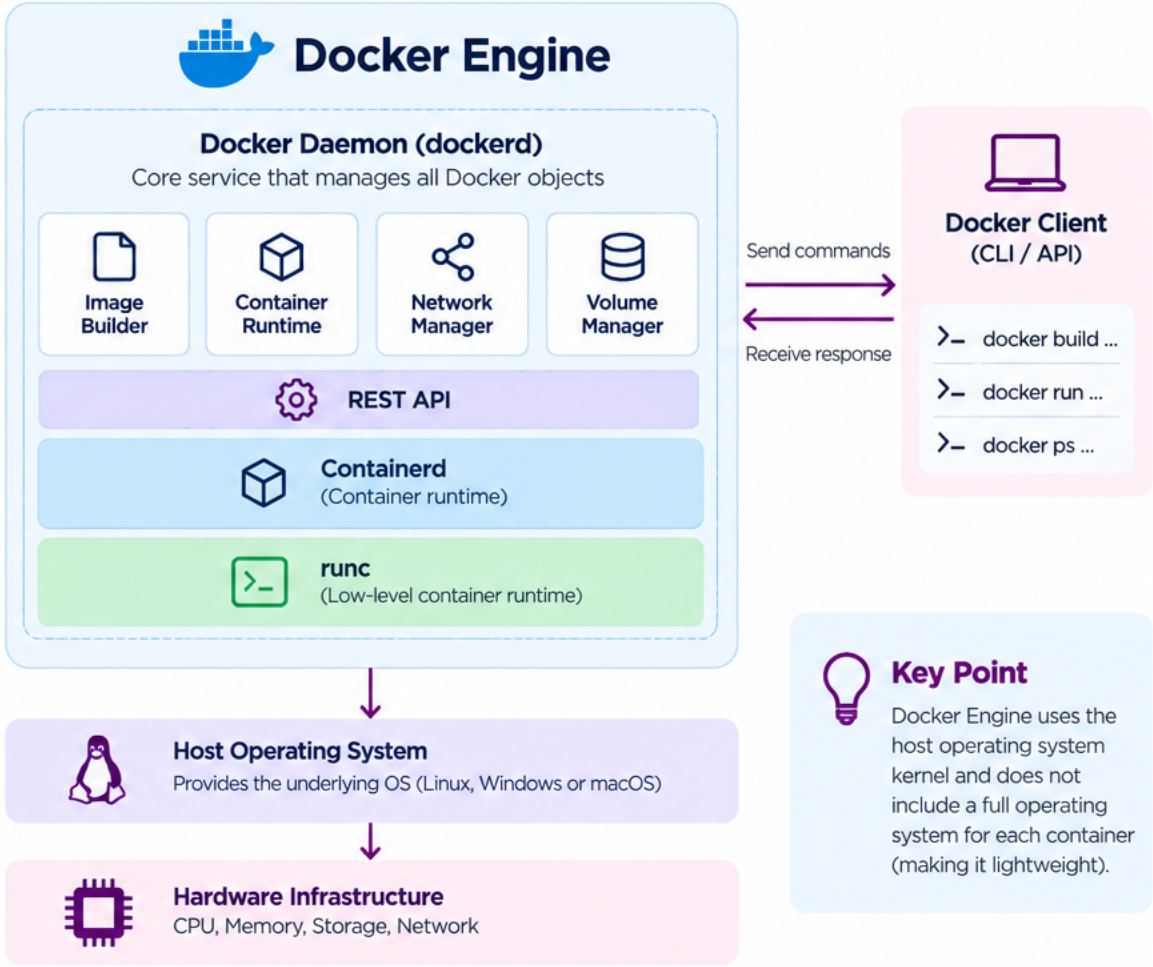
Docker Engine is the core software that builds, runs and manages containers on your machine or server.

Build
Build container images from a Dockerfile or existing images.

Run
Create and run containers from images.

Manage
Manage images, containers, networks and volumes.

Provide Interfaces
Exposes a CLI (docker command) and a REST API for automation and integration.



Key Point
Docker Engine uses the host operating system kernel and does not include a full operating system for each container (making it lightweight).

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Docker CLI

The Docker CLI (Command Line Interface) is a tool that allows you to interact with Docker from your terminal.

You use Docker CLI to build images, run containers, manage containers, images, networks and volumes.

Simple
Easy to learn and use.

Powerful
Manage all Docker resources.

Flexible
Automate tasks with scripts.

Consistent
Works the same across different environments.

Terminal

```
$ docker --version
Docker version 25.0.3, build 4debf41

$ docker ps
CONTAINER ID   IMAGE     COMMAND                  STATUS         PORTS                    NAMES
a1b2c3d4e5f6   nginx     "nginx -g 'daemon of..."  Up 2 minutes   0.0.0.0:80->80/tcp       web
b2c3d4e5f6g7   redis     "docker-entrypoint.s..."  Up 2 minutes   0.0.0.0:6379->6379/tcp   cache

$ docker images
REPOSITORY      TAG         IMAGE ID          CREATED          SIZE
nginx           latest      605c77e624dd      2 weeks ago      192MB
redis           latest      7614ae9453d1      2 weeks ago      138MB

$ docker run -d -p 80:80 nginx
f3a9c2b4e7d1c6f8a9e0b1c2d3e4f5g6h7i8j9k0l1m2n3o4
```

Common Docker CLI Commands

`docker --version`	Check Docker version
`docker help`	Show help information
`docker pull <image>`	Download an image
`docker images`	List local images
`docker build -t <name> .`	Build an image
`docker run <image>`	Run a container
`docker ps`	List running containers
`docker ps -a`	List all containers
`docker stop <id>`	Stop a container
`docker start <id>`	Start a container
`docker rm <id>`	Remove a container
`docker rmi <image>`	Remove an image
`docker logs <id>`	View container logs

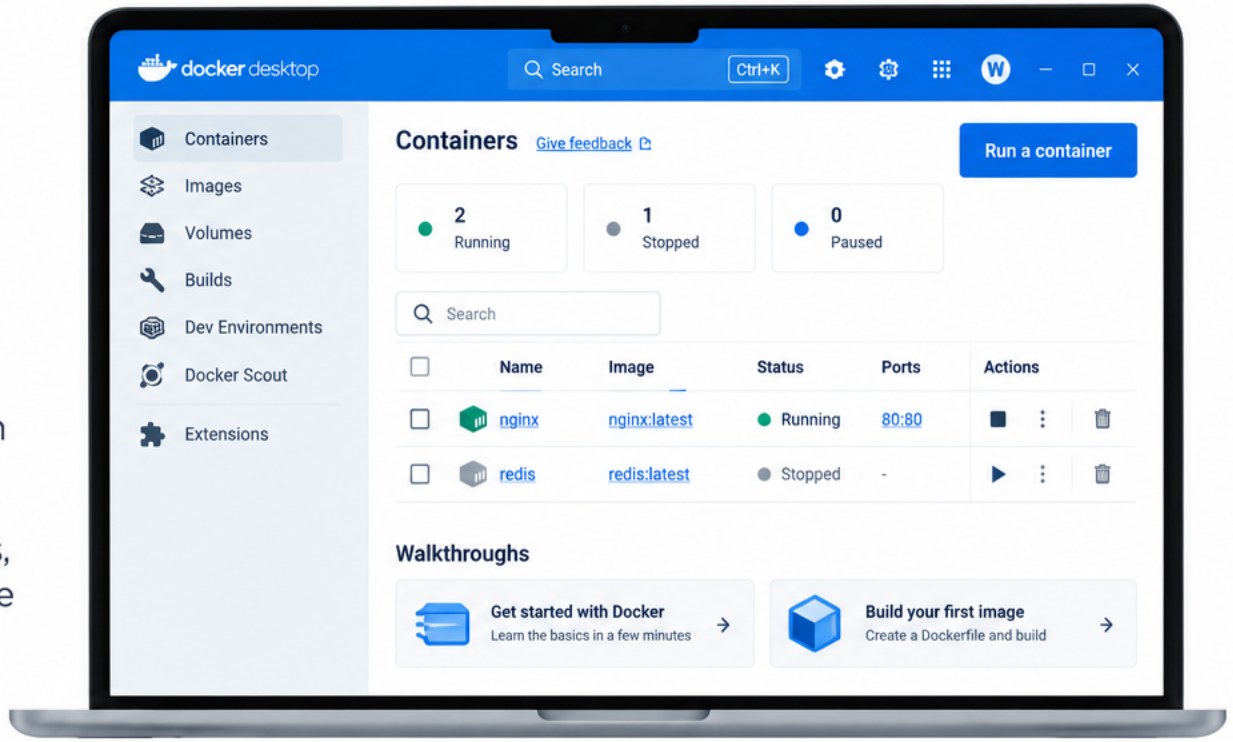
Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Docker Desktop

Docker Desktop is a desktop application that makes it easy to work with Docker on your local machine.

It provides a graphical interface, CLI tools, and built-in tools to build, run and manage containers, images, volumes and more.



Easy to Use
Simple graphical interface for managing Docker.

All-in-One Tool
Includes Docker Engine, CLI, Compose, Kubernetes and other tools.

Built-in Tools
Manage containers, images, volumes, networks and extensions.

Works on Multiple OS
Available on Windows, macOS and Linux.

Great for Learning and Development
Quick setup and a user-friendly interface for hands-on practice.

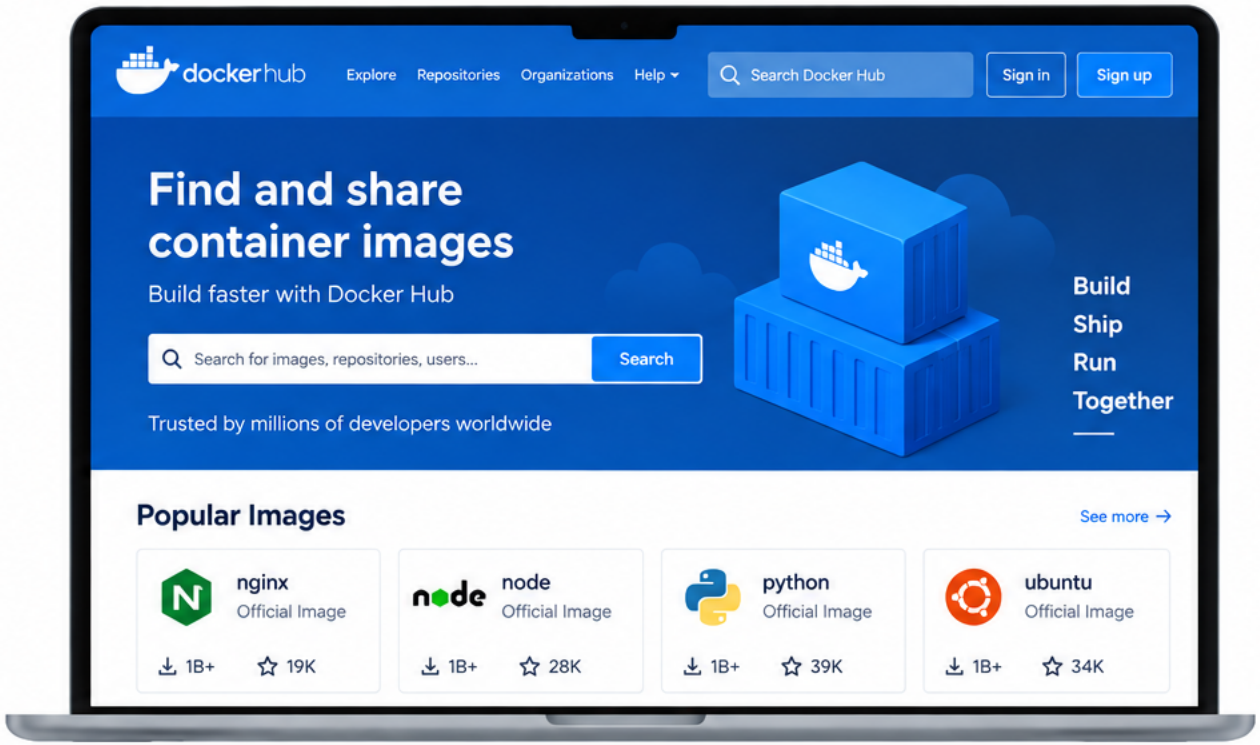
Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Docker Hub

Docker Hub is a cloud-based registry that allows you to store, manage and share container images.

You can pull official images, publish your own images and collaborate with others in the Docker community.



Pull Images
Access thousands of official and community images.

Publish Images
Store and share your own container images.

Collaborate
Work with your team using repositories, organisations and access controls.

Explore
Discover images, base images and tools from the global Docker community.

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Docker Images and Containers

An image is a blueprint. A container is a running instance of that blueprint.

Docker Image

A read-only template used to create containers.

Dockerfile

```
FROM python:3.11
COPY app.py /app/
RUN pip install -r
requirements.txt
CMD ["python",
"app.py"]
```

Instructions to build an image

docker build →

Docker Image
e.g. myapp:1.0

Contains the application code, dependencies and configuration

Key Points about Images

- Images are read-only.
- Built from a Dockerfile or existing images.
- Contain everything needed to run the application (code, libraries, settings).
- Can be stored and shared in a registry (e.g. Docker Hub).
- Used to create one or more containers.

Docker Container

A running instance of an image.

Docker Image
e.g. myapp:1.0

docker run →

Docker Container
Running (Isolated)

A lightweight, isolated environment that runs the application.

Key Points about Containers

- Containers are running instances of an image.
- Each container has its own isolated environment.
- You can run multiple containers from the same image.
- Containers are lightweight and start quickly.
- Changes made inside a container do not affect the original image (unless you create a new image).

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Docker Workflow

From code to a running container

Write your application, build a Docker image, store it in a registry, and run it as a container.

1 Write Application

Your application code and dependencies (e.g. Python, Node.js, Java, etc.)

2 Build Docker Image

Docker reads the Dockerfile and builds an image that includes your application and dependencies.

3 Store in Docker Hub

Push the image to Docker Hub (or a private registry) so it can be shared and accessed from anywhere.

4 Pull Image

Pull the image from Docker Hub to your local machine or server.

5 Run Container

Run a container from the image. The container runs your application in an isolated environment.

6 Access Your Application

Your application is running and accessible through a port.

Example Commands

```
# 1. Build image
docker build -t myapp:1.0 .
```

→

```
# 2. Tag image
docker tag myapp:1.0
username/myapp:1.0
```

→

```
# 3. Push to Docker Hub
docker push
username/myapp:1.0
```

→

```
# 4. Pull image
docker pull
username/myapp:1.0
```

→

```
# 5. Run container
docker run -d -p 8080:80
--name myapp
username/myapp:1.0
```

→

```
# 6. Access app
Open
http://localhost:8080
in your browser
```

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Docker Installation and Setup

Get Docker up and running on your machine in a few simple steps.

Same Experience Across Platforms

Docker gives you a consistent experience on Windows, macOS and Linux.

Windows Installation

Install Docker Desktop on Windows 10/11

1. **Check system requirements**
   Windows 10/11 (64-bit) with virtualization enabled (WSL 2 is recommended).
2. **Download Docker Desktop**
   Go to https://www.docker.com/products/docker-desktop and download the Windows installer.
3. **Run the installer**
   Open the downloaded file and follow the installation wizard.
4. **Start Docker Desktop**
   Launch Docker Desktop from the Start menu.
5. **Verify installation**
   Open a terminal (PowerShell or Command Prompt) and run:

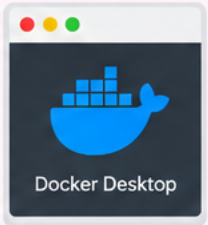
```
docker --version
docker run hello-world
```

macOS Installation

Install Docker Desktop on macOS

1. **Check system requirements**
   macOS 12 (Monterey) or later, with Apple Silicon (M1/M2/M3) or Intel chip.
2. **Download Docker Desktop**
   Go to https://www.docker.com/products/docker-desktop and download the macOS version (.dmg).
3. **Install Docker Desktop**
   Open the .dmg file and drag Docker to the Applications folder.
4. **Start Docker Desktop**
   Open Docker from Applications.
5. **Verify installation**
   Open a terminal and run:

```
docker --version
docker run hello-world
```



Linux Installation

Install Docker Engine on Ubuntu (Example)

1. **Update package index**

```
sudo apt update
```

2. **Install required packages**

```
sudo apt install apt-transport-https ca-certificates
curl software-properties-common
```

3. **Add Docker's official GPG key**

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg |
sudo gpg --dearmor -o /etc/apt/trusted.gpg.d/docker.gpg
```

4. **Add Docker repository**

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=
/etc/apt/trusted.gpg.d/docker.gpg] https://download.docker.com/linux/ubuntu
$(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list
```

5. **Install Docker Engine**

```
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io
```

6. **Start and verify**

```
sudo systemctl start docker
sudo systemctl enable docker
docker --version
docker run hello-world
```

Verify Successful Installation

You should see output similar to this:

```
$ docker --version
Docker version 25.0.3, build 4debf41

$ docker run hello-world
Hello from Docker!
```

Next Steps

- Explore the Docker Desktop dashboard (Windows/macOS)
- Learn basic Docker commands
- Pull and run your first container
- Build your own image using a Dockerfile
- Explore Docker Hub

You are now ready to build, run and ship applications with Docker.

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Working with Docker Containers

Use simple commands to create, run, manage and remove containers.

Containers are lightweight, isolated environments that run your application consistently anywhere.

1 Run a Container

Start a container from an image.

```
docker run -d -p 80:80 \
--name web nginx
```

`-d` Run in background (detached)
`-p` Map port (host:container)
`--name` Give the container a name

2 List Containers

View running containers.

```
docker ps
```

Use **`docker ps -a`** to see all containers (including stopped ones).

3 View Container Details

Get more information about a container.

```
docker inspect web
```

Replace **`web`** with your container name or container ID.

4 View Logs

See container output.

```
docker logs web
docker logs -f web
```

Use **`-f`** to follow logs in real time.

5 Access a Running Container

Open a shell inside a container.

```
docker exec -it web bash
```

Use **`docker exec`** to run a command inside a running container.

6 Stop, Start and Remove

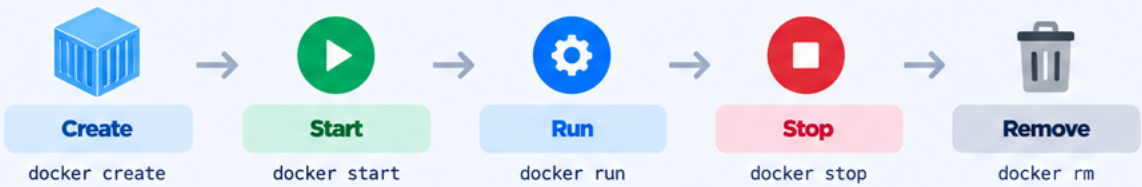
Manage the container lifecycle.

```
docker stop web
docker start web
docker rm web
```

Use **`docker rm -f web`** to force remove a running container.

Container Lifecycle

A container goes through different states during its lifecycle.



Useful Commands

Common container operations.

`docker ps`	List running containers
`docker ps -a`	List all containers
`docker logs <name>`	View container logs
`docker exec -it <name> bash`	Open a shell in container
`docker stop <name>`	Stop a container
`docker start <name>`	Start a container
`docker rm <name>`	Remove a container
`docker rm -f <name>`	Force remove a running container

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Practical: Run Nginx in Docker

Pull the Nginx image, run it in a container and access it from your browser.

Goal

Run Nginx in a Docker container and view the default Nginx page in your browser.

1 Check Docker is installed

Open a terminal and check the Docker version.

```
docker --version
Docker version 25.0.3, build 4debf41
```

2 Pull the Nginx image

Download the latest Nginx image from Docker Hub.

```
docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
...
Status: Downloaded newer image for nginx:latest
```

3 Run the container

Run Nginx and map port 80 on your machine to port 80 in the container.

```
docker run -d -p 80:80 --name my-nginx nginx
a3f5e8c9d7b1e4c2f6a8b9e0d1c2f3a4b5c6d7e8
```

4 Check the container is running

List running containers to confirm.

```
docker ps
CONTAINER ID   IMAGE   COMMAND                  PORTS
a3f5e8c9d7b1   nginx   "/docker-entrypoint.."   0.0.0.0:80->80/tcp
```

5 Access Nginx in your browser

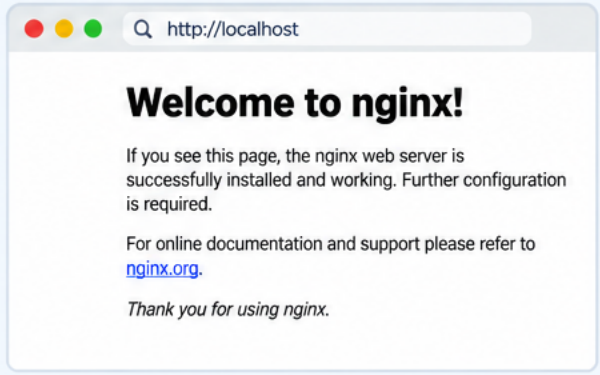
Open your web browser and go to:

http://localhost

You should see the default Nginx welcome page.

6 Nginx default page

If everything is working, you will see a page like this in your browser.



Useful Commands

Stop the container	Start the container	Remove the container	View container logs
`docker stop my-nginx`	`docker start my-nginx`	`docker rm my-nginx`	`docker logs my-nginx`

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Docker Images

A Docker image is a read-only template used to create containers.

An image contains everything your application needs to run (code, runtime, libraries, dependencies and configuration).

1 What is a Docker Image?

- A read-only template.
- Used to create containers.
- Contains the application code, runtime, libraries, dependencies and configuration.
- Images can be pulled from a registry (e.g. Docker Hub) or built from a Dockerfile.

2 Image vs Container

Image

- Read-only template
- Used to create containers
- Stored in a registry
- Does not change while running

Container

- Running instance of an image
- Has a writable layer
- Can be started, stopped, removed
- Can have multiple containers from the same image

3 Image Source and Flow

Build | Store (e.g. Docker Hub) | Run

Write a Dockerfile → Build an image → Store or share → Run a container

4 Common Docker Image Commands

Command	Description
`docker pull <image>`	Download an image from a registry
`docker images`	List local images
`docker rmi <image>`	Remove an image
`docker build -t <name> .`	Build an image from a Dockerfile
`docker tag <image> <name>`	Tag an image
`docker push <image>`	Push an image to a registry

5 Examples

Pull an image from Docker Hub

```
$ docker pull nginx
```

List local images

```
$ docker images
REPOSITORY   TAG       IMAGE ID       SIZE
nginx        latest    a1b2c3d4e5f6   192MB
```

Remove an image

```
$ docker rmi nginx
```

6 Popular Official Images

Image	Use Case
`nginx`	Web server
`httpd`	Web server (Apache)
`python`	Python runtime
`node`	Node.js runtime
`openjdk`	Java runtime
`mysql`	MySQL database
`postgres`	PostgreSQL database
`redis`	In-memory database
`ubuntu`	Base operating system

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Building Images with Dockerfile

Use a Dockerfile to package your application into a Docker image.

A Dockerfile is a text file with instructions to build a Docker image automatically.

1 Create a simple application

Create a project folder with a simple web app.

```
myapp/
├── app.py
├── requirements.txt
└── Dockerfile
```

2 Add application code

Create a simple Python Flask app.

app.py (Python)

```python
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return "<h1>Hello from Docker!</hh1>"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

3 Create a Dockerfile

Write instructions to build the image.

Dockerfile (Dockerfile)

```dockerfile
# Use an official Python image
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Copy requirements and install dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY app.py .

# Expose the application port
EXPOSE 5000

# Command to run the application
CMD ["python", "app.py"]
```

4 Build the image

Run the build command in the project folder.

```
$ docker build -t myapp:1.0 .

Sending build context to Docker daemon  5.12kB
...
Successfully built 1a2b3c4d5e6f
Successfully tagged myapp:1.0
```

5 Verify the image

List your local images.

```
$ docker images

REPOSITORY  TAG     IMAGE ID      SIZE
myapp       1.0     1a2b3c4d5e6f  152MB
```

6 Run the container

Run the image and map the port.

```
$ docker run -d -p 5000:5000 \
  --name myapp-container myapp:1.0
d4e5f6a7b8c9
```

Test in your browser

http://localhost:5000

Hello from Docker!

Key Points

- A Dockerfile automates the image creation process.
- Use an official base image (e.g. python, nginx, node).
- Copy your code and install dependencies.
- Expose the required port.
- Build with `docker build`.
- Run with `docker run`.
- The same image can run anywhere (development, testing or production).

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Containerization of Django Applications

Package your Django application into a Docker container and run it consistently anywhere.

Build once. Run anywhere.
Docker makes it easy to package your Django application with all its dependencies.

1 Create a Django Project

Create a new Django project and app.

```
django-admin startproject myproject
cd myproject
python manage.py startapp core
```

2 Project Structure

Your project folder should look like this.

```
myproject/
├── manage.py
├── myproject/
├── core/
├── requirements.txt
├── Dockerfile
└── .dockerignore
```

3 Create requirements.txt

List your project dependencies.

```
Django==5.0.2
gunicorn==21.2.0
```

4 Create a Dockerfile

Write instructions to build the image.

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["gunicorn", "myproject.wsgi:application",
    "--bind", "0.0.0.0:8000"]
```

5 Create .dockerignore

Exclude unnecessary files.

.dockerignore

```
__pycache__/
*.pyc
*.pyo
*.pyd
.env
db.sqlite3
media/
staticfiles/
.git
venv/
```

6 Build the Docker image

Run the build command in the project folder.

```
$ docker build -t myproject:1.0 .
Sending build context to Docker daemon...
Step 1/7 : FROM python:3.11-slim
...
Successfully built a1b2c3d4e5f6
Successfully tagged myproject:1.0
```

7 Run the container

Start the container and map the port.

```
$ docker run -d -p 8000:8000 \
   --name django-app myproject:1.0
a1b2c3d4e5f6...
```

8 Access in your browser

Open your browser and go to:

http://localhost:8000

Useful Docker Commands

List images	`docker images`	Remove container	`docker rm django-app`
List containers	`docker ps`	Remove image	`docker rmi myproject:1.0`
Stop container	`docker stop django-app`	View logs	`docker logs django-app`

Next Steps

- Use environment variables for sensitive settings
- Add PostgreSQL or any database using Docker Compose
- Collect static files for production
- Use Docker Compose for multi-container setup (Django + database)
- Deploy to a cloud platform (AWS, Azure, etc.)

Key Takeaways

- Docker packages your Django app and its dependencies.
- It ensures consistency across development, testing and production.
- Use a slim base image to keep images small.
- Exclude unnecessary files with .dockerignore.
- The same container can run anywhere.

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Containerizing a Node.js Application

Package your Node.js app into a Docker container and run it anywhere.

Build. Ship. Run.
Docker makes it easy to package your Node.js application with all its dependencies.

1 Create a Node.js Project

Create a simple Express app.

```
mkdir node-docker-app
cd node-docker-app
npm init -y
npm install express
```

2 Add Application Code

Create a file named app.js.

app.js

```
const express = require('express');
const app = express();

const port = process.env.PORT || 3000;

app.get('/', (req, res) => {
  res.send('Hello from Node.js in Docker!');
});

app.listen(port, () => {
 console.log(`Server running on port ${port}`);
});
```

3 Create package.json

Already created with npm init -y.

```
{
  "name": "node-docker-app",
  "version": "1.0.0",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  },
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

4 Create a Dockerfile

Write instructions to build the image.

Dockerfile

```
FROM node:20-alpine
WORKDIR /app
# Copy package files and install dependencies
COPY package*.json ./
RUN npm install --production
# Copy application code
COPY . .
# Expose the port
EXPOSE 3000
# Start the application
CMD ["node", "app.js"]
```

5 Create .dockerignore

Exclude unnecessary files.

.dockerignore

```
node_modules
npm-debug.log
.git
.env
Dockerfile
.dockerignore
```

6 Build the Docker image

Run the build command in the project folder.

```
$ docker build -t node-app:1.0 .
Sending build context to Docker daemon...
Step 1/7 : FROM node:20-alpine
...
Successfully built a1b2c3d4e5f6
Successfully tagged node-app:1.0
```

7 Run the container

Start the container and map the port.

```
$ docker run -d -p 3000:3000 \
  --name node-app node-app:1.0
d4e5f6a7b8c9e0f1
```

8 Access in your browser

Open your browser and go to:

http://localhost:3000

Useful Docker Commands

List images	docker images	Remove container	docker rm node-app
List containers	docker ps	Remove image	docker rmi node-app:1.0
Stop container	docker stop node-app	View logs	docker logs node-app

Next Steps

- Use environment variables for configuration
- Use Docker Compose for multi-container setup (e.g. with a database like MongoDB or PostgreSQL)
- Set up a production build (multi-stage Dockerfile)
- Deploy to a cloud platform (AWS, Azure, etc.)

Key Takeaways

- A Docker image packages your app and its dependencies.
- Use an official base image (e.g. node:20-alpine).
- Keep images small with .dockerignore and production installs.
- The same container can run anywhere.
- Great for development, testing and deployment.

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Containerizing a Spring Boot Application

Package your Spring Boot application into a Docker container and run it anywhere.

Same application. Any environment. With Docker.
Build once and run anywhere.

1 Create a Spring Boot Project

Generate a new Spring Boot project (using Spring Initializr or your IDE).

```
# Using Spring Initializr (example)
# or create from your IDE
mkdir springboot-docker-app
cd springboot-docker-app
```

2 Project Structure

Your project folder should look like this.

```
springboot-docker-app/
  src/
    main/
    test/
  pom.xml
  Dockerfile
  .dockerignore
  README.md
```

3 Build the Application

Build the Spring Boot application using Maven.

```
# Build the application
mvn clean package -DskipTests
```

This will create a **.jar** file in the **target/** directory.

4 Create a Dockerfile

Write instructions to build the image.

Dockerfile

```
FROM eclipse-temurin:17-jdk-alpine
WORKDIR /app

COPY target/*.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

5 Create .dockerignore

Exclude unnecessary files.

.dockerignore

```
target/
.mvn/
mvnw
mvnw.cmd
.idea/
*.log
.git/
```

6 Build the Docker Image

Run the build command in the project folder.

```
$ docker build -t springboot-app:1.0 .

Sending build context to Docker daemon...
Step 1/5 : FROM eclipse-temurin:17-jdk-alpine
...
Successfully built a1b2c3d4e5f6
Successfully tagged springboot-app:1.0
```

7 Run the Container

Start the container and map the port.

```
$ docker run -d -p 8080:8080 \
  --name springboot-app springboot-app:1.0
f8e3c2d4a6b7...
```

8 Test in Your Browser

Open your browser and go to:

http://localhost:8080

http://localhost:8080

Hello from Spring Boot!

Your application is now running in a Docker container.

Useful Docker Commands

List images	`docker images`	Remove container	`docker rm springboot-app`
List containers	`docker ps`	Remove image	`docker rmi springboot-app:1.0`
Stop container	`docker stop springboot-app`	View logs	`docker logs springboot-app`

Next Steps

- Use environment variables for configuration.
- Externalize application.properties.
- Use a multi-stage build for smaller images.
- Push the image to Docker Hub or a private registry.
- Deploy to cloud (AWS, Azure, etc.).

Key Takeaways

- Docker packages your Spring Boot application with all its dependencies.
- It ensures consistency across development, testing and production.
- Use a lightweight base image (e.g. eclipse-temurin:17-jdk-alpine).
- Keep your images small and secure.
- The same container can run anywhere.

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Containerizing a .NET Application

Package your .NET application into a Docker container and run it anywhere.

.NET + Docker

Same application. Any environment. With Docker.
Build once and run anywhere.

1 Create a .NET Project

Create a new ASP.NET Core Web API project.

```
# Using .NET CLI
dotnet new webapi -n DotNetApp
cd DotNetApp
```

2 Project Structure

Your project folder should look like this.

```
DotNetApp/
├── Controllers/
├── Program.cs
├── appsettings.json
├── DotNetApp.csproj
├── Dockerfile
└── .dockerignore
```

3 Build the Application

Test that your application runs locally.

```
dotnet restore
dotnet build
dotnet run
```

Your application should be running on http://localhost:5000

4 Create a Dockerfile

Write instructions to build the image.

Dockerfile (Dockerfile)

```
# Build stage
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY *.csproj ./
RUN dotnet restore
COPY . .
RUN dotnet publish -c Release -o /app/publish
# Runtime stage
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
WORKDIR /app
COPY --from=build /app/publish .
EXPOSE 8080
ENTRYPOINT ["dotnet", "DotNetApp.dll"]
```

5 Create .dockerignore

Exclude unnecessary files.

.dockerignore

```
bin/
obj/
.vs/
.git/
*.user
*.suo
*.log
Dockerfile
.dockerignore
```

6 Build the Docker Image

Run the build command in the project folder.

```
$ docker build -t dotnetapp:1.0 .
Sending build context to Docker daemon...
Step 1/10 : FROM mcr.microsoft.com/dotnet/sdk:8.0
...
Successfully built a1b2c3d4e5f6
Successfully tagged dotnetapp:1.0
```

7 Run the Container

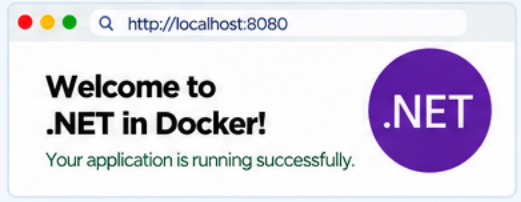
Start the container and map the port.

```
$ docker run -d -p 8080:8080 \
  --name dotnetapp dotnetapp:1.0
f8e3c2d4a6b7...
```

8 Access in Your Browser

Open your browser and go to:

http://localhost:8080



Useful Docker Commands

List images	`docker images`
List containers	`docker ps`
Stop container	`docker stop dotnetapp`
View logs	`docker logs dotnetapp`
Remove container	`docker rm dotnetapp`

Next Steps

- Use environment variables for configuration.
- Connect to a database (e.g. SQL Server) using connection strings.
- Use multi-stage builds for smaller images.
- Push the image to Docker Hub or a private registry.
- Deploy to a cloud platform (Azure, AWS, etc.).

Key Takeaways

- Docker packages your .NET application with all its dependencies.
- It ensures consistency across development, testing and production.
- Use official .NET base images.
- Keep your images small and secure.
- The same container can run anywhere.

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Docker Storage

Understand how Docker stores and manages data in containers.

Keep your data persistent, flexible and portable with Docker storage.

1 Why Storage Matters

By default, data inside a container is temporary. If the container is stopped or removed, the data can be lost.

Docker storage helps you keep data persistent and share it across containers.

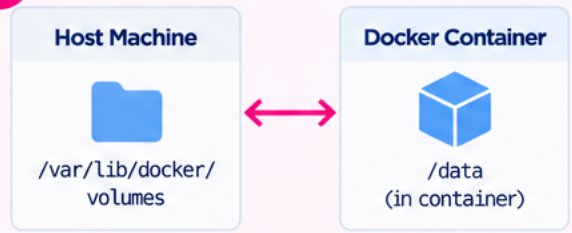
2 Types of Docker Storage

Volumes
Managed by Docker (recommended)

Bind Mounts
Map a host directory to a container

tmpfs Mounts
Stored in the host memory (temporary)

3 How It Works



Docker stores volumes on the host machine and mounts them inside the container.

4 Using a Volume

Create a volume and attach it to a container.

```
# Create a volume
docker volume create mydata
# Run a container with the volume
docker run -d \
  --name myapp \
  -v mydata:/data \
  nginx
```

5 Using a Bind Mount

Map a host directory to a container.

```
# Run a container with a bind mount
docker run -d \
  --name myapp \
  -v /path/on/host:/data \
  nginx
```

6 Using tmpfs

Store data in memory (non-persistent).

```
# Run a container with tmpfs mount
docker run -d \
  --name myapp \
  --tmpfs /data \
  nginx
```

7 Managing Storage

Useful commands.

`docker volume ls`	List all volumes
`docker volume inspect <name>`	View volume details
`docker volume rm <name>`	Remove a volume
`docker system df`	View disk usage
`docker system prune`	Remove unused data

8 Use Cases

Databases
Persist database data (e.g. MySQL, PostgreSQL)

Application Data
Store user uploads, logs and files

Configuration
Share configuration files across containers

Development
Mount source code during development

9 Key Takeaways

- Containers are ephemeral, storage makes data persistent.
- Use volumes for most use cases.
- Bind mounts are useful for development.
- tmpfs is for temporary, in-memory storage.
- Manage storage with Docker commands.
- Choose the right storage type based on your needs.

Same application. Persistent data. Anywhere.

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Docker Networking

Understand how containers communicate with each other and with the outside world.

Connect containers.
Build better applications.
With Docker.

1 Why Networking Matters

Containers are isolated by default, but they need to communicate with:

- Other containers
- Services on the host machine
- External services (internet)

Docker networking makes this simple and secure.

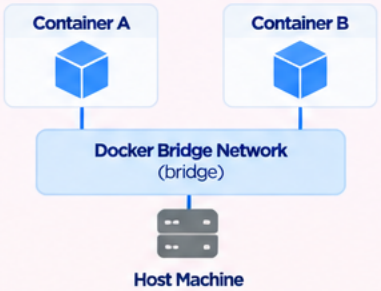
2 Docker Network Types

Docker provides different network drivers for different use cases.

Network Type	Purpose
bridge	Default network. Good for single host container communication.
host	Container uses the host's network stack.
none	No networking (fully isolated).
overlay	For multi-host communication (e.g. Docker Swarm).
macvlan	Assign a MAC address to a container (on physical network).

3 Default Bridge Network

When you run a container, it is attached to the default bridge network.



4 Custom Bridge Network

Create a user-defined network to allow containers to communicate by name.

```
# Create a custom network
docker network create mynet

# Run two containers on the network
docker run -d --name app1 --network mynet nginx
docker run -d --name app2 --network nynet nginx

# Test connectivity
# (from app1 ping app2)
docker exec -it app1 ping app2
```

5 Container to Host Communication

Use special DNS name or host networking.

```
# From a container, access a service
# running on the host machine
docker run -it --rm alpine
ping host.docker.internal

# Or use host network (Linux only)
docker run -it --network host nginx
```

6 Container to External Network

Containers can access the internet by default (using NAT).

Docker uses the host's network (NAT) to allow containers to reach external services.

7 Port Mapping

Expose container ports to the host.

```
# Run an Nginx container and map port 80
docker run -d --name web -p 8080:80 nginx

# Access in browser
http://localhost:8080

# View port mapping
docker ps
```

8 Inspecting Networks

Useful commands to manage and troubleshoot networks.

```
# List networks
docker network ls

# Inspect a network
docker network inspect mynet

# Remove a network
docker network rm  mynet
```

Key Takeaways

- Containers can communicate using Docker networks.
- Use user-defined networks for better control.
- Port mapping allows access from the host.
- Docker provides different network drivers for different scenarios.
- Use network inspection tools for troubleshooting.

Common Use Cases

Web Applications
Connect frontend and backend containers.

Databases
Containers communicate by name.

Microservices
Multiple services on the same network.

External APIs
Access third-party services from containers.

Same network.
More possibilities.
Build, connect and scale with Docker.

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Environment Variables and Configuration

Keep your Docker containers **flexible**, secure and easy to manage.

Same image. Different environments.
Use environment variables and configuration files to adapt your application without changing the image.

1 What are Environment Variables?

Environment variables are key–value pairs that provide configuration to your application at runtime.

Example:

```
DB_HOST=localhost
DB_PORT=5432
APP_ENV=development
```

2 Using -e with docker run

Pass environment variables directly when starting a container.

```
docker run -d --name myapp \
  -e DB_HOST=postgres \
  -e DB_PORT=5432 \
  -e APP_ENV=production \
     myapp:1.0
```

3 Using an --env-file

Store variables in a file and pass it to the container.

.env file

```
DB_HOST=postgres
DB_PORT=5432
APP_ENV=production
SECRET_KEY=your_secret_key
API_URL=https://api.example.com
```

```
docker run -d \
  --name myapp \
  --env-file .env \
  myapp:1.0
```

4 Using Docker Compose

Define environment variables in docker-compose.yml.

docker-compose.yml

```
services:
  web:
    image: myapp:1.0
    environment:
      - DB_HOST=postgres
      - DB_PORT=5432
      - APP_ENV=production
    env_file:
      - .env
```

5 Using ENV in a Dockerfile

Set default values that can be overridden at runtime.

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app

ENV APP_ENV=production
ENV PORT=8000

COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

6 Accessing Environment Variables in Code

Your application reads the variables from the environment.

Python | Node.js | Spring Boot | .NET

```
import os
db_host = os.getenv("DB_HOST")
db_port = os.getenv("DB_PORT", "5432")
app_env = os.getenv("APP_ENV", "development")
print(f"Database host: {db_host}")
```

7 Best Practices

- Keep sensitive values out of your image.
- Use .env files (do not commit to Git).
- Use different values for development, testing and production.
- Use secrets management for sensitive data (e.g. Docker Secrets, AWS Secrets Manager).
- Provide sensible defaults using ENV in the Dockerfile.

8 Common Use Cases

- Database credentials
- Application settings
- API keys
- Third-party service URLs
- Feature toggles

Key Takeaway

Use environment variables and configuration to build one image that works across multiple environments.

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Docker Compose

Define and run multi-container applications with a single configuration file.

Multiple containers.
One file.
A simpler workflow.
Build, run and manage complex applications with Docker Compose.

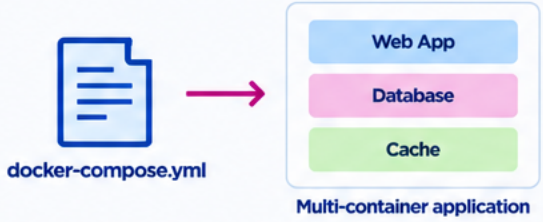
1 What is Docker Compose?

Docker Compose is a tool for defining and running multi-container applications using a YAML file.

- Define services, networks and volumes in one file.
- Start, stop and manage all containers with simple commands.
- Ideal for development, testing and local environments.

2 How It Works

You define your application in a docker-compose.yml file, then use a single command to start all the services.



3 Example: docker-compose.yml

A simple example with a web app (Django), PostgreSQL and Redis.

docker-compose.yml

```yaml
version: '3.8'
services:
  web:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DEBUG=1
      - DATABASE_URL=postgres://postgres:password@db:5432/mydb
    depends_on:
      - db
      - redis
  db:
    image: postgres:15
    environment:
      - POSTGRES_DB=mydb
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=password
    volumes:
      - db_data:/var/lib/postgresql/data
  redis:
    image: redis:7
volumes:
  db_data:
```

4 Common Commands

Manage your application with simple commands.

`docker compose up`	Start all services
`docker compose up -d`	Start in detached mode
`docker compose down`	Stop and remove containers
`docker compose ps`	View running containers
`docker compose logs`	View logs
`docker compose build`	Build or rebuild images
`docker compose exec web bash`	Access a container shell

5 Benefits

Simpler Management
Define and run multiple containers with one file.

Consistent Environments
Ensure the same setup across all machines.

Faster Development
Spin up your entire stack quickly.

Works for Different Stacks
Use with Python, Node.js, Java, .NET and more.

6 Use Cases

- Local development environments
- Microservices applications
- Testing and CI/CD pipelines
- Team collaboration
- Full-stack applications

7 Key Takeaways

- Docker Compose simplifies multi-container setup.
- Use a YAML file to define your services.
- Manage your application with simple commands.
- It is widely used for local development and testing.
- It works with different technologies and stacks.

Focus on building your application. Let Docker Compose handle the rest.

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

3-Tier Application with Docker Compose

Run the complete application stack (frontend, backend and database) using a single docker-compose file.

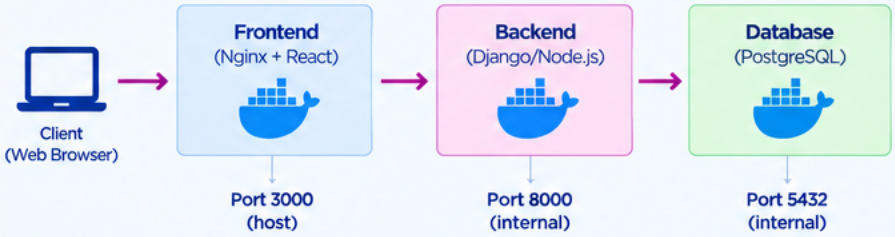
Multiple containers. One application. Real-world architecture.
Build, run and manage a 3-tier application with Docker Compose.

1 What is a 3-Tier Application?

A 3-tier application separates the application into three layers:

- **Presentation Layer (Frontend)**
  User interface (e.g. React)
- **Application Layer (Backend)**
  Business logic (e.g. Django, Node.js, Spring Boot, or .NET)
- **Data Layer (Database)**
  Data storage (e.g. PostgreSQL or MySQL)

2 Architecture Overview



3 Project Structure

```
three-tier-app/
├── docker-compose.yml
├── frontend/
│   ├── Dockerfile
│   └── ... (React code)
├── Backend/
│   ├── Dockerfile
│   └── ... (Django/Node.js code)
└── db/
    └── init.sql (optional)
```

4 docker-compose.yml

docker-compose.yml

```yaml
version: '3.8'
services:
  db:
    image: postgres:15
    container_name: app-db
    environment:
      POSTGRES_DB: myapp
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
    volumes:
      - db_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  backend:
    build: ./backend
    container_name: app-backend
    depends_on:
      - db
    environment:
      DB_HOST: db
      DB_NAME: myapp
      DB_USER: myuser
      DB_PASSWORD: mypassword
    ports:
      - "8000:8000"

  frontend:
    build: ./frontend
    container_name: app-frontend
    depends_on:
      - backend
    ports:
      - "3000:80"

volumes:
  db_data:
```

5 Key Dockerfiles (Examples)

Frontend (React)

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build
FROM nginx:alpine
COPY --from=0 /app/build /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

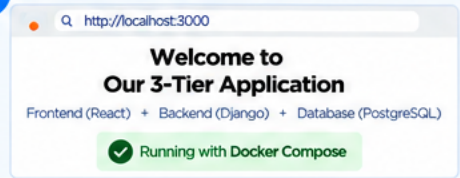
Backend (Django)

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["gunicorn", "myproject.wsgi:application",
     "--bind", "0.0.0.0:8000"]
```

6 Run the Application

```
# Build and start all services
docker compose up -d --build

# View running containers
docker compose ps

# View logs
docker compose logs -f

# Stop the application
docker compose down
```

7 Access the Application



8 Key Takeaways

- Docker Compose manages multi-container applications.
- Each tier runs in its own container.
- Services can communicate using service names (e.g. db, backend).
- It is easy to start, stop and manage the entire stack.
- This setup is ideal for local development and testing.
- The same approach can be used in production with changes.

Real-world skills for real opportunities.
Build. Practise. Grow with Tech365.

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Optimising Docker Image

Build smaller, faster and more secure Docker images.

Smaller images. Faster deployments. A more secure application.

1 Why Optimise?

Optimising your Docker images helps you:

- Reduce image size
- Improve build speed
- Lower storage and bandwidth costs
- Decrease security vulnerabilities
- Achieve faster deployments

2 Use a Smaller Base Image

Choose minimal base images (e.g. Alpine or slim).

Larger image	Smaller image
`FROM node:18`	`FROM node:18-alpine`
Image size: ~900MB	Image size: ~200MB

3 Use Multi-Stage Builds

Build your application in stages and copy only what you need.

```
# Build stage
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Production stage
FROM node:18-alpine
WORKDIR /app
COPY --from=builder /app/dist ./dist
CMD ["node", "dist/index.js"]
```

4 Reduce Image Layers

Combine related commands to minimise layers.

Not optimised

```
RUN apt-get update
RUN apt-get install -y curl
RUN apt-get clean
```

Multiple layers

Optimised

```
RUN apt-get update && \
    apt-get install -y curl && \
    rm -rf /var/lib/apt/lists/*
```

Single layer

5 Use .dockerignore

Exclude unnecessary files from the build context.

.dockerignore

```
node_modules
dist
.git
Dockerfile
*.log
.env
```

Build only what you need.

6 Remove Unnecessary Files

Keep only the files required to run the application.

- Remove build tools and cache files
- Exclude tests, logs and docs
- Clean package manager caches

7 Use Minimal Dependencies

Install only production dependencies.

```
npm install --only=production
or
npm ci --omit=dev
```

Smaller image.
Fewer vulnerabilities.

8 Choose the Right Format

Use efficient image formats and tools.

- Docker
- BuildKit (faster builds)
- Multi-arch images (when needed)

9 Key Takeaways

- Use minimal base images.
- Use multi-stage builds.
- Minimise layers.
- Use .dockerignore.
- Remove unnecessary files.
- Install only what you need.
- Build faster, smaller and more secure images.

Optimise today. Deploy faster tomorrow.

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Docker Security

Build, run and deploy containers securely.

Secure containers.
Safer applications.
Stronger outcomes.

1 Why Docker Security Matters

Containers are lightweight and fast, but security is essential to:

- Protect your applications and data
- Prevent unauthorised access
- Reduce the risk of supply chain attacks
- Meet compliance requirements
- Build trust with your users

2 Use Trusted Base Images

Start from official and minimal images.

Avoid

```
FROM ubuntu:latest
```

Large image, unknown updates.

Better

```
FROM node:20-alpine
```

Smaller, maintained and more secure.

3 Keep Images Updated

Regularly update base images and packages.

```
# Update packages in Dockerfile
RUN apt-get update && \
    apt-get upgrade -y && \
    rm -rf /var/lib/apt/lists/*
```

Rebuild images regularly to include security updates.

4 Run as Non-Root User

Do not run containers as root.

```
# Create a non-root user
RUN useradd -m appuser
USER appuser
```

Limit privileges to reduce the impact of a potential breach.

5 Use Minimal Permissions

Give only the permissions a container needs.

```
# Run container with read-only filesystem
docker run --read-only \
  --cap-drop=ALL \
  --security-opt no-new-privileges \
  myapp:1.0
```

6 Manage Secrets Securely

Do not hardcode secrets in images.

Avoid

```
ENV DB_PASSWORD=12345
ENV API_KEY=abcd
```

Better

```
# Use Docker secrets
docker secret create \
  db_password ./db.txt
```

Use Docker secrets or environment variables injected at runtime.

7 Scan Images for Vulnerabilities

Identify and fix security issues early.

```
# Scan image with Trivy
trivy image myapp:1.0
```

Use tools like Trivy, Snyk or Docker Scout to scan your images and dependencies.

8 Secure the Container Runtime

Use security features provided by Docker.

- Use content trust (image signing)
- Enable user namespaces
- Use seccomp and AppArmor profiles.
- Limit container capabilities
- Use resource limits (CPU, memory)

9 Monitor and Audit

Keep track of what is running and detect unusual activity.

- Use Docker logs and monitoring tools
- Audit image sources and changes
- Set up alerts for suspicious behaviour

10 Follow Docker Security Best Practices

- Use official and minimal base images
- Keep images and packages updated
- Run containers as non-root users
- Use least privilege and drop unnecessary capabilities
- Manage secrets securely
- Scan images for vulnerabilities
- Monitor logs and set up alerts

Secure by Default. Build with Confidence.

Apply Docker security best practices and build safer, more reliable applications.

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Container Monitoring and Security

Keep your containers healthy, secure and reliable in production.

Observe.
Detect.
Secure.
Run with confidence.

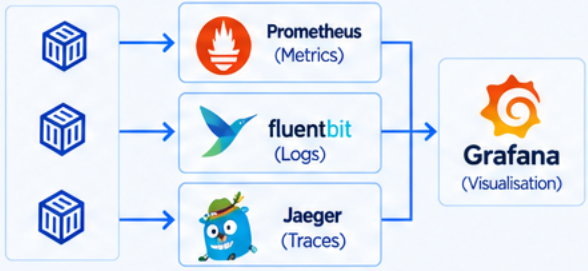
1 Why It Matters

Monitoring and security help you:

- Keep applications running smoothly
- Detect and fix issues early
- Prevent security vulnerabilities
- Improve reliability and performance
- Meet compliance requirements

2 Monitoring Architecture

Collect metrics, logs and traces from your containers.



3 Monitor Container Metrics

Use Prometheus and Grafana to collect and visualise metrics.

```yaml
# docker-compose.yml (monitoring stack)
services:
  prometheus:
    image: prom/prometheus
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
  grafana:
    image: grafana/grafana
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin
```

Prometheus
Collects and stores metrics.

Grafana
Visualises metrics with dashboards.

4 Monitor Container Logs

Use Fluent Bit (or Loki) to collect and centralise logs.

```yaml
# docker-compose.yml (Fluent Bit)
services:
  fluent-bit:
    image: fluent/fluent-bit
    volumes:
      - ./fluent-bit.conf:/fluent-bit/etc/fluent-bit.conf
    ports:
      - "2020:2020"  # metrics
      - "24224:24224" # log input
```

fluentbit

Collects container logs and forwards them to your logging platform (e.g. Loki, Elasticsearch).

5 Trace and Analyse Requests

Use Jaeger for distributed tracing in microservices.

Jaeger

- Track requests across services
- Find performance bottlenecks
- Understand dependencies

6 Scan Images for Vulnerabilities

Use Trivy to scan container images.

```
# Scan a local image
trivy image myapp:1.0

# Scan an image from a registry
trivy image nginx:latest

# Scan filesystem (e.g. Dockerfile)
trivy fs .
```

Trivy
Finds OS packages and application vulnerabilities.

7 Secure Your Containers

Follow these security best practices.

- Use trusted base images
- Run as non-root user
- Use least privilege permissions
- Scan images regularly
- Use secrets (not hardcoded values)
- Limit network access
- Keep images updated

8 Set Alerts

Get notified before small issues become big problems.

- Set up alert rules in Prometheus
- Integrate with email, Slack or Teams
- Monitor container health, CPU, memory, errors and restarts

Key Takeaway

A well-monitored and secure container environment keeps your applications stable, safe and ready to scale.

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Docker in CI/CD

Automate build, test and deployment with Docker.

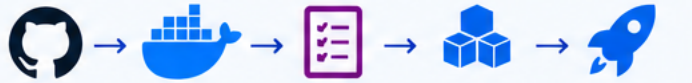
Build.
Test.
Deploy.
Faster with Docker.

1 Why Use Docker in CI/CD?

- Consistent build environments
- Faster and more reliable deployments
- Easy dependency management
- Works the same from development to production
- Supports automated testing
- Reduces environment-related issues

2 Typical CI/CD Pipeline with Docker

Code commit → Build image → Test → Push → Deploy



Code Commit | Build Docker Image | Run Tests (in container) | Push to Registry (e.g. Docker Hub, ECR) | Deploy (to server or cloud)

3 Example: GitHub Actions Workflow

Automate build, test and push with Docker.

.github/workflows/docker.yml

```yaml
name: Build and Push Docker Image
on:
  push:
    branches: [ main ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build Docker image
        run: docker build -t myapp:latest .
      - name: Run tests
        run: docker run --rm myapp:latest npm test
      - name: Push to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKER_USERNAME }}
          password: ${{ secrets.DOCKER_PASSWORD }}
      - run: docker push myapp:latest
```

4 Run Tests in a Docker Container

Ensure your application works before deployment.

```
# Build the image
docker build -t myapp:latest .

# Run tests inside the container
docker run --rm myapp:latest npm test
```

Same tests.
Same environment.
Fewer surprises.

5 Push Images to a Registry

Store your images for deployment.

```
# Login to Docker Hub
docker login

# Tag and push
docker tag myapp:latest
   yourname/myapp:latest
docker push yourname/myapp:latest
```

Docker Hub
(or AWS ECR, Azure ACR, etc.)

6 Deploy with Docker

Deploy your container image to a server or cloud.

- Deploy to VM or bare metal with Docker
- Deploy to Kubernetes
- Deploy to AWS (ECS / EKS)
- Deploy to Azure (ACI / AKS)

7 Best Practices

- Use multi-stage builds to keep images small
- Run tests in the pipeline
- Use image tagging (e.g. version, commit hash)
- Scan images for vulnerabilities
- Store secrets securely (GitHub Secrets, etc.)
- Automate deployment after successful tests
- Use Docker Compose or Kubernetes for multi-container apps

8 Key Takeaway

Docker makes your CI/CD pipeline consistent, faster and more reliable from code to production.

Build once. Deploy anywhere.

Tech365
Leveraging technology for growth

LEARN | PRACTICE | APPLY

Docker in AWS

Build, store and run your containerised applications in the cloud.

Same containers. More possibilities. Deploy, scale and run on AWS.

1 Why Use Docker in AWS?

- Consistent environments from development to production
- Easy deployment and scaling
- Better resource utilisation
- Supports modern architectures (microservices)
- Integrates with AWS services
- Improves reliability and security

2 Architecture Overview

Build, store and run Docker containers on AWS.

3 Build and Push Image to Amazon ECR

Build your image and push it to AWS ECR.

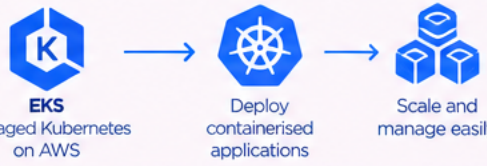
```
# Authenticate to ECR
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin \
  123456789012.dkr.ecr.us-east-1.amazonaws.com

# Build the image
docker build -t myapp:latest .

# Tag the image
docker tag myapp:latest \
  123456789012.dkr.ecr.us-east-1.amazonaws.com/myapp:latest

# Push to ECR
docker push 123456789012.dkr.ecr.us-east-1.amazonaws.com/myapp:latest
```

4 Run Containers on Amazon ECS

Use ECS to run and manage your containers.

5 Run Containers on Amazon EKS

Use EKS to run Kubernetes with your Docker containers.



6 Other Useful AWS Services

Services that work well with Docker containers.

7 Example: Deploy to ECS with Fargate

Run your Docker containers without managing servers.

```
{
  "family": "myapp",
  "networkMode": "awsvpc",
  "containerDefinitions": [{
    "name": "myapp",
    "image": "123456789012.dkr.ecr.us-east-1.amazonaws.com/myapp:latest",
    "portMappings": [{"containerPort": 80}],
    "memory": 512,
    "cpu": 256
}
```

Use a task definition to configure your container.

8 Best Practices

- Use private ECR repositories
- Apply IAM least privilege
- Use ECS Fargate or EKS (no server management)
- Enable logging and monitoring (CloudWatch)
- Use environment variables and AWS Secrets Manager
- Scan images for vulnerabilities
- Keep images small and up to date
- Use load balancer and auto scaling
- Follow AWS Well-Architected best practices

9 Key Takeaway

Docker and AWS help you build, deploy and scale applications faster, more securely and more reliably.

Same skills. Bigger opportunities.
Build today for a better tomorrow.

Tech365 INNOVATION HUB

YAML BASICS

Human Friendly Data Serialization Language

docker

YAML (YAML Ain't Markup Language) is a readable data serialization format commonly used for configuration files, APIs, and DevOps tools like Docker, Kubernetes, Ansible, and more.

WHAT IS YAML?

- Human-readable data format
- Uses indentation (not brackets)
- Based on key: value pairs
- Supports complex data structures

Easy to read, write, and maintain by humans.

YAML SYNTAX RULES

- Indentation is everything (use spaces, not tabs)
- Key and value are separated by : (colon)
- Use consistent indentation (2 spaces is common)
- Case sensitive
- Supports comments using #
- No commas

BASIC DATA TYPES

```
string: "hello" or 'hello' or hello
number: 10, 3.14
boolean: true, false
null: null or ~
list (array):
  - item1
  - item2
map (dictionary):
  key: value
```

YAML EXAMPLE

```
# This is a comment
app:
  name: Tech365
  version: 1.0
  active: true
  port: 8080
features:
  - easy_to_read
  - human_friendly
  - powerful

database:
  host: localhost
  user: admin
  password: secret123
```

INDENTATION EXAMPLE

Correct (2 spaces)

```
server:
  host: localhost
  port: 80
  ssl:
    enabled: true
    cert: /path/cert.pem
```

Incorrect (Wrong Indentation)

```
server:
 host: localhost
  port: 80
 ssl:
    enabled: true
    cert: /path/cert.pem
```

YAML uses indentation to understand structure. Wrong indentation can lead to errors.

COMPLEX EXAMPLE

```
services:
  web:
    image: nginx:latest
    ports:
      - "80:80"
    environment:
      - ENV=production
      - DEBUG=false
    depends_on:
      - db
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: mydb
    volumes:
      - db_data:/var/lib/postgresql/data
volumes:
  db_data:
```

COMMON USE CASES

- **Docker Compose** — Define multi-container applications
- **Kubernetes** — Define deployments, services, etc.
- **Ansible** — Configuration management and automation
- **Configuration Files** — Application settings and environment
- **CI/CD Pipelines** — Define build, test and deployment steps

YAML VS JSON

	YAML	JSON
Human Readable	Yes (easy)	No (compact)
Syntax	Indentation based	Braces {} and brackets []
Comments	Supported	Not supported
File Extension	.yml or .yaml	.json
Use Case	Config files, DevOps	APIs, Data Exchange

INLINE (FLOW) STYLE

```
# Same data in flow style
user: { name: John, age: 30, active: true }

languages: ["Python", "Java", "Go"]
servers: [ {host: localhost, port: 80}, {host: example.com, port: 443} ]
```

Flow style is compact but less readable for large data. Block style (indentation) is preferred.

QUICK TIPS

- Always use spaces, never tabs.
- Keep indentation consistent.
- Use meaningful keys.
- Add comments to explain complex parts.
- Validate your YAML using online tools or IDE extensions.

Validate YAML

- yamllint
- onlineyamltools.com
- VS Code YAML extension

234 802 970 4730 | info@tech365.ng | www.tech365.ng | @tech365ng